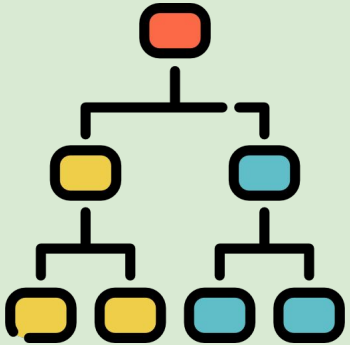
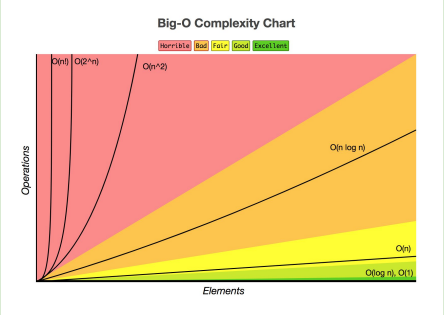
Lesson 6

Algorithms, Data Structure





agilaghamirzayev

Algorithms and Data Structure

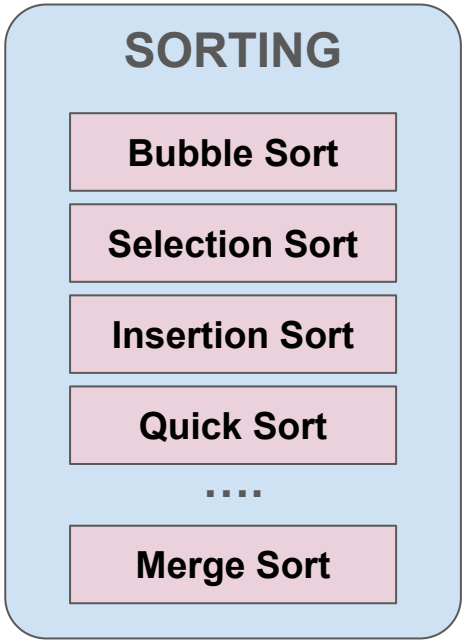
Algorithms is about how to solve different problems, often by searching through and manipulating data structures.

Data Structures is about how data can be stored in different structures.

Algorithms and Data Structure

Algorithms is about how to solve different problems, often by searching through and manipulating data structures.

Data Structures is about how data can be stored in different structures.



in agilaghamirzayev

Algorithms and Data Structure

Algorithms is about how to solve different problems, often by searching through and manipulating data structures.

Data Structures is about how data can be stored in different structures.

SORTING

- Bubble Sort
- Selection Sort
- Insertion Sort
- Quick Sort
- ....
- Merge Sort

SEARCHING

- Linear Search
- Binary Search

in agilaghamirzayev

Algorithms and Data Structure

agilaghamirzayev

Basic Coding For Beginners		Data Structures & Algorithms	
Topics		Companies	Sheets
Arrays	0%	Searching	0%
Two Pointers	0%	Linked Lists	0%
Stacks & Queues	0%	Hashing	0%
Backtracking	0%	Binary Trees	0%
BST, Heaps & Map	0%	Math & Bit Manipulation	0%
Dynamic Programming	0%	Greedy Algorithm	0%
Graphs	0%	String & Tries	0%

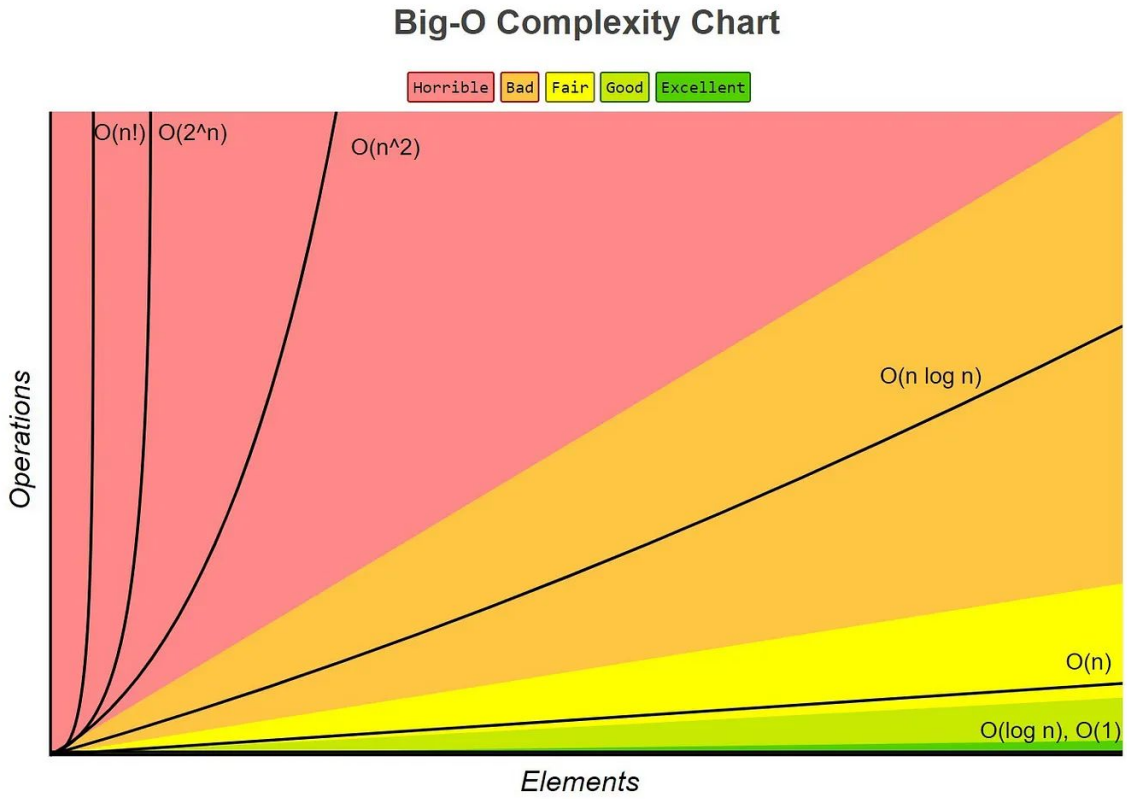
https://workat.tech/problem-solving/practice

Algorithms and Data Structure

Time Complexity describes how the **execution time of an algorithm grows** as the input size (**n**) increases.

- It's not actual time (seconds)
- It's about **growth rate**

We express it using **Big-O notation**.



Algorithms and Data Structure

Complexity	Name	Intuition
O(1)	Constant	Same time no matter input
O(log n)	Logarithmic	Reduce problem by half
O(n)	Linear	Loop through all
O(n log n)	Linearithmic	Sorting
$O(n^2)$	Quadratic	Nested loops
$O(2^n)$	Exponential	Bruteforce combinations

Algorithms and Data Structure

```
String[] userCache = new String[1000000];

public String getUser(int id) {
    return userCache[id]; // O(1)
}
```

O(1)

Algorithms and Data Structure

```
public int binarySearch(int[] arr, int target) {
    int left = 0, right = arr.length - 1;

    while (left <= right) {
        int mid = (left + right) / 2;

        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}
```

Searching in sorted data (Binary Search)

O(log n)

Algorithms and Data Structure

```
public int sumTransactions(int[] transactions) {
    int sum = 0;

    for (int t : transactions) {
        sum += t;
    }

    return sum;
}
```

O(n)

Algorithms and Data Structure

agilaghamirzayev

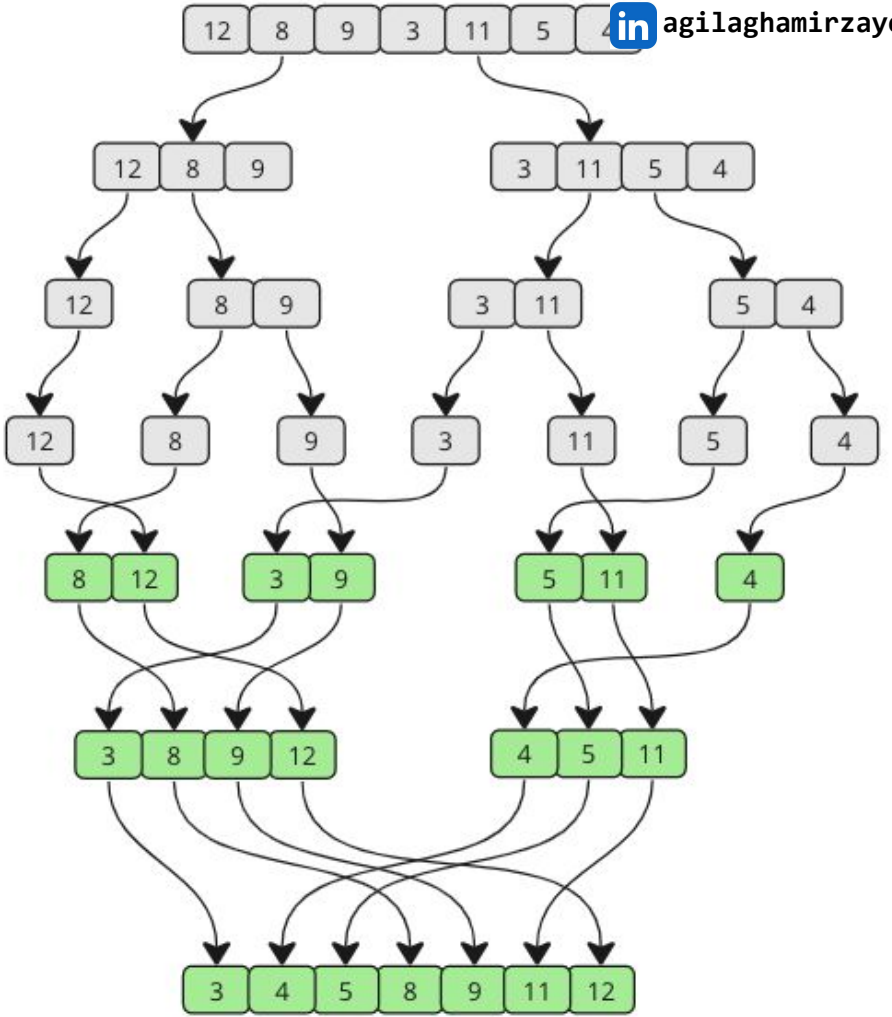
```
public void mergeSort(int[] arr, int left, int right) {
    if (left >= right) return;

    int mid = (left + right) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}

private void merge(int[] arr, int left, int mid, int right) {
    int[] temp = new int[right - left + 1];

    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }

    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    for (int p = 0; p < temp.length; p++) {
        arr[left + p] = temp[p];
    }
}
```

O(n log n)

Algorithms and Data Structure

agilaghamirzayev

```
public void mergeSort(int[] arr, int left, int right) {
    if (left >= right) return;

    int mid = (left + right) / 2;

    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);

    merge(arr, left, mid, right);
}

private void merge(int[] arr, int left, int mid, int right) {
    int[] temp = new int[right - left + 1];

    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }

    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    for (int p = 0; p < temp.length; p++) {
        arr[left + p] = temp[p];
    }
}
```



O(n log n)

Algorithms and Data Structure

agilaghamirzayev

```
public void compareUsers(List<String> users) {
    for (int i = 0; i < users.size(); i++) {
        for (int j = 0; j < users.size(); j++) {
            System.out.println(users.get(i) + " vs " + users.get(j));
        }
    }
}
```

O(n^2)

https://www.bigocheatsheet.com
https://roadmap.sh/datastructures-and-algorithms